

轮 趣 科 技

WHEELTEC Self-Balancing Bicycle Circuit and Programming Manual

Recommended to follow our public number for updates



Imprint:

releases	dates	Description of content
V1.0	2022/06/23	First Release

preamble

This document mainly explains the circuit and software development design ideas and implementation of WHEELTEC Z100 Servo Self-Balancing Bicycle. The circuit design includes the description and principle of each electronic module, and the software development includes the distribution of STM32 microcontroller resource design and the principle of implementation of each part of the program.

Finally, the flowchart of the bicycle balance control is provided, and the part about the principle of LQR balance control is within another document [3. Self-balancing unmanned bicycle based on handlebar steering control.docx].

catalogs

Preamble.....	2
1. Circuit Design	4
1.1 Microcontroller Minimum System Design.....	4
1.2 Gyroscope Circuit Design.....	4
1.3 Motor Drive Circuit Design.....	5
1.4 Motor encoder	6
1.5 Potentiometer Circuit Design	7
1.6 Servo Circuit Design	8
1.7 Power Supply Circuit Design	8
2. Software Development.....	9
2.1 Trolley Master Programming.....	9
2.2 Interrupt Task Programming for Trolley Control.....	10
2.3 Motor and Servo Driver Design.....	10
2.4 Motor Encoder Reading Programming.....	11
2.5 Bluetooth Programming.....	12
2.6 ADC Acquisition of Battery Voltage with Potentiometer Programming	13
3. Trolley Control Flowchart	14

1. circuit design

1.1 Microcontroller Minimum System Design

The servo cart adopts STM32F103C8T6 as the main control, and its schematic diagram is shown in Figure 1-1. The input voltage of the main control is 5V or 3.3V, and the power supply part adopts AMS1117 chip, which converts 5V to 3.3V for the STM32 chip and external modules. In addition, the main control board is equipped with a one-key download serial interface circuit, a user key and a reset key, a programmable LED and a power indicator.

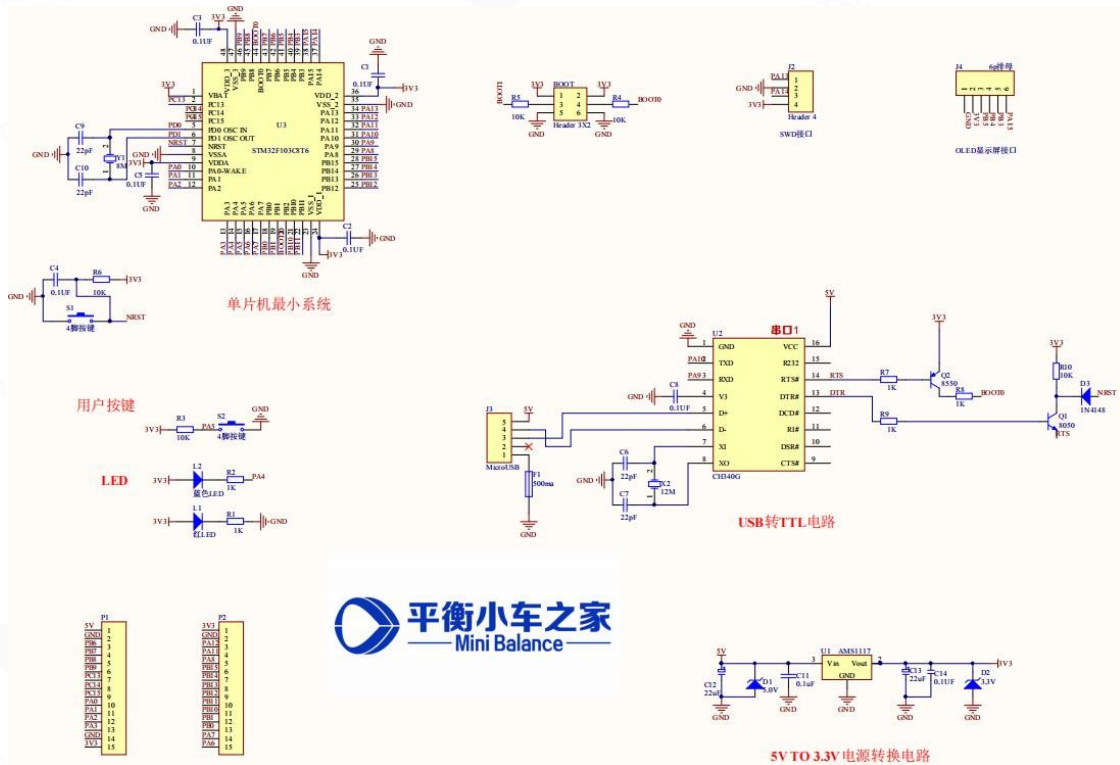


Figure 1-1 STM32 Minimum System Design

1.2 Gyroscope Circuit Design

The cart uses MPU6050 chip as gyro sensor, which has 3-axis angular velocity and 3-axis acceleration sensors, and uses IIC protocol to communicate, and its internal digital motion processor (DMP) hardware acceleration engine can directly output 6-axis attitude solution data; in order to satisfy the needs and the accuracy of the measurements, the sensor provides different ranges, and the range of the gyro sensor can be from $\pm 250^\circ$ to $\pm 2000^\circ$, and the range of the accelerometer can be from $\pm 2g$ to

$\pm 16g$, which can satisfy most of the usage scenarios. The gyroscope range is from $\pm 250^\circ$ to $\pm 2000^\circ$ and the accelerometer range is from $\pm 2g$ to $\pm 16g$, which can meet most of the usage scenarios. The schematic diagram is shown in Figure 1-2. Which AD0 pin can change the chip in the IIC communication address, AD0 access GND address for 0x68, access VCC for 0x69; INT pin is the chip's interrupt pin, when the chip data ready, the pin will be changed from a high level to a low level;

The data-ready frequency is the same as the sampling frequency configured by the chip; SDA and SCL are IIC communication pins, where the It is connected to the normal IO of the microcontroller, so the program will be written to communicate with the chip by simulating an IIC.

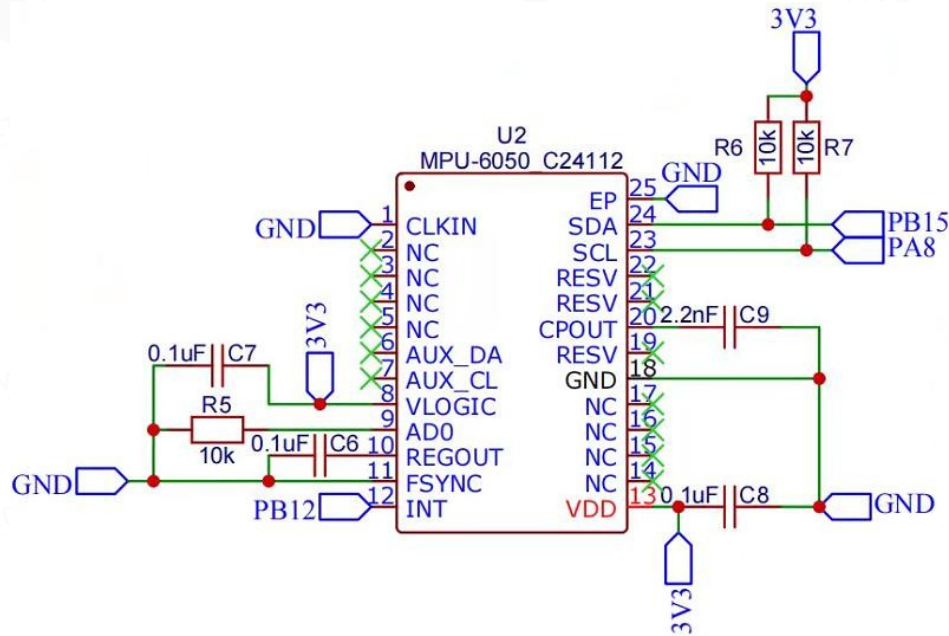
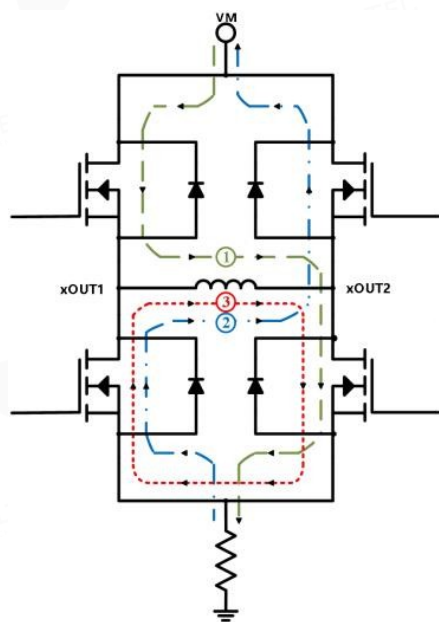


Figure 1-2 MPU6050 Connection Diagram

1.3 Motor Drive Circuit Design

The standard version of servo trolley only uses one motor, which is used to drive the rear wheel of the trolley. The driver chip adopts



AT8236, which is an H-bridge structure composed of 4 mos tubes, as shown in Figure 1-3.

Figure 1-3 AT8236 Internal H-Bridge Schematic

We can perform closed loop control of the motor by reading the pulse counts and thus obtaining the speed of the motor.

where the number of pulses is related to the motor reduction ratio as follows:

Number of pulses output from the encoder for one revolution of the motor = motor reduction ratio x encoder accuracy (number of lines)

In the program, we use the timer of STM32 to read the number of pulses output from the encoder, which has the advantage that it does not need to take up the resources of the CPU, but is read by the hardware, and can be configured as a 4-fold frequency, which improves the accuracy of the encoder. So we need to pin out the two pins of the timer and provide power for the encoder, the schematic diagram is shown in Figure 1-6.

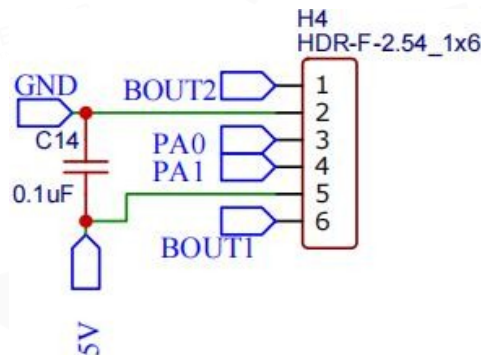


Figure 1-6 Motor Encoder and Positive/Negative Interface

1.5 Potentiometer Circuit Design

The servo cart contains two potentiometers, one to adjust the initial position of the cart's servo and one to adjust the cart's median balance. The purpose of adjusting both potentiometers is to balance the trolley without re-modifying the program, which is convenient for the user. The schematic diagram is shown in Figure 1-7. The potentiometer used in the figure has a maximum resistance of 10k, and a 10k resistor is connected in series, so the voltage on the potentiometer ranges from 0 to 1.65V.

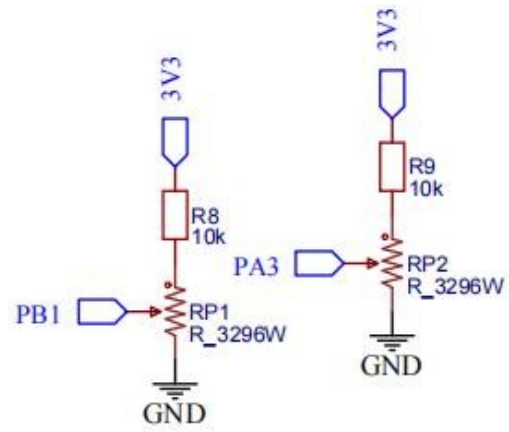


Figure 1-7 Potentiometer Schematic

1.6 Servo circuit design

The servo driver only needs one way PWM to complete the control, in which the relationship between PWM duty cycle and servo rotation angle is shown in Figure 1-8 below. It should be noted that the PWM frequency of the servo drive is generally 50Hz.

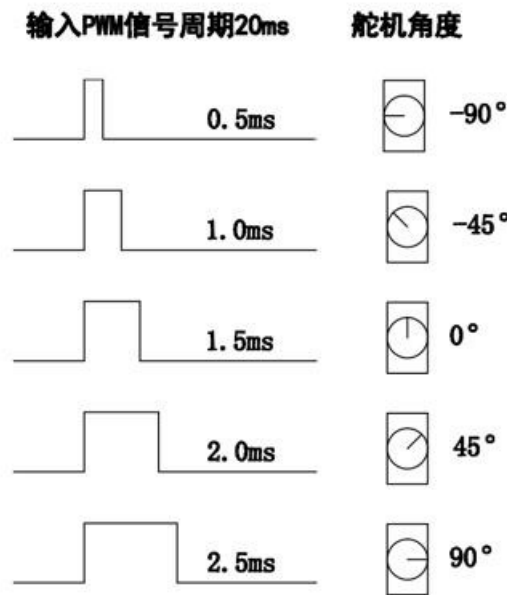


Figure 1-8 Input Servo PWM Duty Cycle versus Angle

Therefore the control of the servo only needs to lead the power line and a timer pin. The schematic diagram is shown in Figure 1-9.

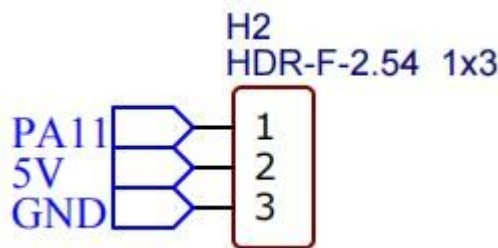


Figure 1-9 Servo Interface Schematic

1.7 Power supply circuit design

Cart power supply part of the RT8289GSP buck chip is used to regulate the voltage, the chip supports a wide voltage input of 5 ~ 32V, after the chip regulator can be 5V5A external power output; In addition, the power supply input in series with a 10k and a 1k resistor, the battery voltage voltage divider to facilitate the microcontroller to carry out the sampling and calculation, and its schematic diagram as shown in Figure 1-10. It should be noted that the power supply voltage will be input to the

motor driver chip and then apply the voltage to the motor driver chip.

Add on the motor, the voltage applied to the trolley motor should not be too high, it is recommended that the voltage range is 7~12 V. On the trolley Powered by a 2s flight model battery, 8.4V at full charge.

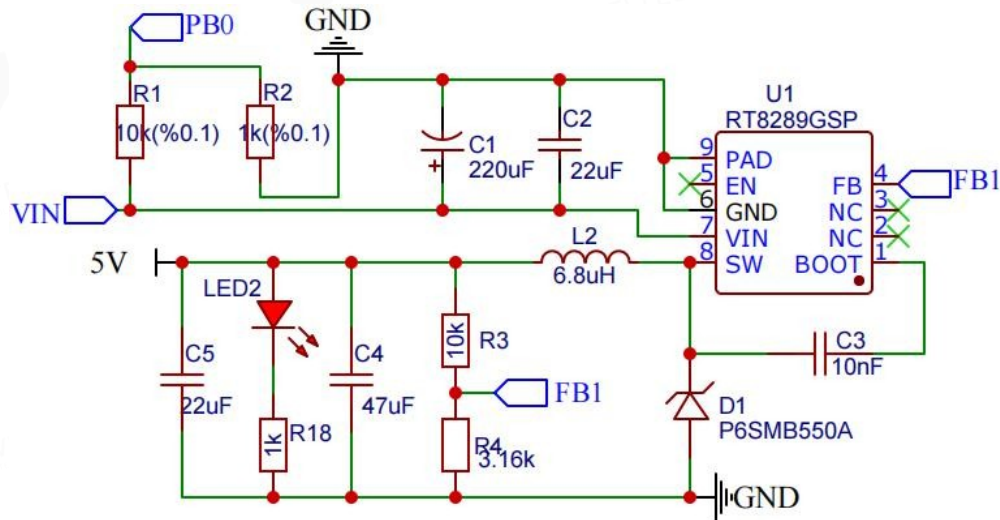


Figure 1-10 Power Supply Circuit Schematic

2. software development

2.1 Trolley Master Programming

The initialization of all peripherals of the cart is executed in the main program, and the microcontroller resources used are explained below:

USART1: Used to send data to the host computer and display waveforms

USART3: Used for mutual communication between the Bluetooth app and the trolley

TIM4: Using TIM4CH1 and TIM4CH2 to generate PWM to drive the rear wheel motor

TIM2: Initialized to quadrature encoder mode, used to capture the number of pulses output from the motor encoder ADC: Uses channels 3, 8, and 9 of ADC1, used to capture the potentiometer that modifies the mechanical median, respectively,

Battery voltage, potentiometer to modify servo initial position

TIM1: Use TIM1CH4 to generate PWM to drive the servo.

EXTI: Configure PB12 to map to external interrupt line 12, triggered by the INT pin of the MPU6050 for 5ms.

external interrupt

IIC: the car does not use hardware IIC, but through the PA8, PB15 to simulate the IIC communication

SPI: No hardware SPI is used by the cart, it is emulated by PB3, 4, 5 and PA15 for OLEDs.

demonstrate

SWD: JTAG function is disabled, SWD debugging function is enabled, online debugging and program downloading are possible.

After initializing the above microcontroller resources, the dead loop function is started. In the dead loop it is passed through the flag. The display mode of the trolley is determined by two modes: **app** data sending + **oled** refresh and waveform display by the host computer, and the mode switching is done by pressing a key. In addition, when Bluetooth sends a power-down save command, the data write flag bit in the dead loop will be set to 1, and then the **pid** parameter of the cart will be written to Flash to realize the power-down save function.

2.2 Interrupt Task Programming for Trolley Control

The control interrupt of the cart is provided by the external interrupt triggered by the **MPU6050**, which is based on the following principle: Under the premise of configuring the external interrupt of the **STM32**, after setting the acquisition frequency of the **MPU6050** and turning on the interrupt function of the **MPU6050**, the **INT** pin of the **MPU6050** will trigger the external interrupt of the **STM32** according to the acquisition frequency.

Configuration of the external interrupt is done in **MiniBalance_EXTI_Init()** of the **main** function, and the frequency of the interrupt can be modified by the following macro definition. The location is in the **MPU6050.c** file.

```
#define DEFAULT_MPU_HZ ( 200)
```

During the **5ms** external interrupt task, the microcontroller collects the battery voltage, collects the number of the two potentiometers, and collects the number of the battery voltage.

data, obtaining trolley attitude information, reading encoder data, and performing **PID+LQR** with these data.

Calculate and finally output the calculation results to the motor and servo.

2.3 Motor and Servo Driver Design

In subsection 1.3, we know the motor truth table and motor drive schematic, we need to configure **TIM4CH1** and **TIM4CH2** as **PWM** output mode to control the motor forward and reverse rotation and speed. The initialization process is in the **MiniBalance_PWM_Init(7199,0)** function of the

main function, and the specific configuration process will not be expanded here. The result of the configuration is that channels 1 and 2 of TIM4 are configured as **PWM1** mode, and the frequency of the **PWM** is 10KHz, where 0~7200 corresponds to the duty cycle of the **PWM** from 0~100%.

In subsection 1.6, the control mode of servo and the control interface of servo is known as **PA11**, so it is necessary to configure **TIM1CH4** into **PWM** output mode, which is configured by **Servo_PWM_Init(9999,144)** in the **main** function, and the result of the configuration is that the **PA11** outputs **PWM** signals with the frequency of about 50Hz, and 0~10000 corresponds to the duty cycle of 0~100%. 0~10000 corresponds to 0~100% duty cycle.

After the initialization of the timer is completed, the following according to the truth table of the motor driver began to write the motor forward and reverse control as well as speed control program and servo control program. The following is the realization process:

PWMA1 corresponds to PB7, PWMA2 corresponds to PB6, and SERVO corresponds to PA11.

The definitions and modifications are the respective PWM duty cycles to

```
void Set_Pwm(int moto1)
{
    if(moto1>0)      PWMA1=7200,      PWMA2=7200-
    else             PWMA2=7200,      moto1.
                    PWMA1=7200+moto1;

    SERVO=Servo + SERVO_BIAS;    //Servo
    control
}
```

realize motor speed control and servo angle control.

2.4 Motor encoder reading program design

As the encoder characteristics are known in [subsection 1.4](#), we need a timer to count the number of pulses in the encoder AB. From the encoder interface schematic diagram, we know that we need to configure TIM2 to be in quadrature encoder mode, and we need to pay attention to the fact that the timer only has encoder counting function for channel 1 and channel 2. The configuration process is realized in `Encoder_Init_TIM2()` of the `main` function, and the result is that the TIM2 timer is configured into Encoder Mode 3, which means that the encoder counts every pulse edge, thus realizing a 4-fold frequency and improving the encoder accuracy. Figure 2-1 below demonstrates how the microcontroller's timer is reading encoder pulses.

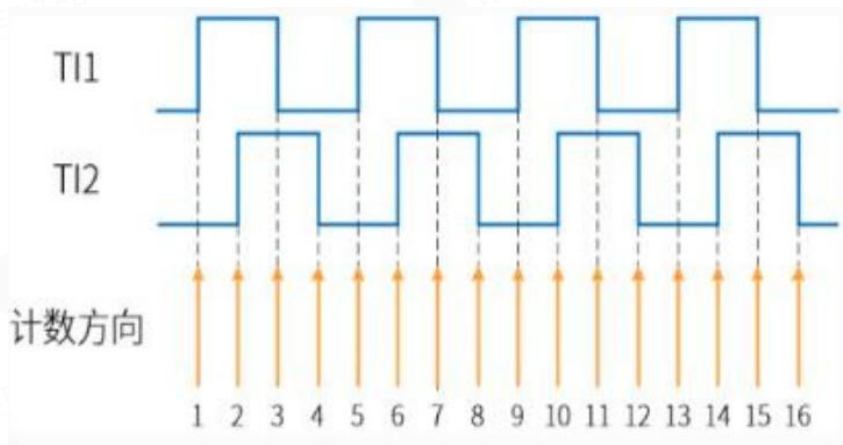


Figure 2-1 Timer Reading Encoder Pulse Diagram

Note: When the direction of rotation of the motor is changed, the phase difference between TI1 and TI2 will also be changed and the counting direction will be changed.

After initialization, the pulse counting of the encoder is done by the timer hardware, we don't need to write any program, we just need to read out

```
int Read_Encoder(u8 TIMX)
{
    int Encoder_TIM;
    switch(TIMX)
```

the value of the counter in the timer, the following is the implementation process:

```
{
    case 2.  Encoder_TIM= (short)TIM2 -> CNT;    TIM2 -> CNT=0;break;
    case 3.  Encoder_TIM= (short)TIM3 -> CNT;    TIM3 -> CNT=0;break;
    case 4.  Encoder_TIM= (short)TIM4 -> CNT;    TIM4 -> CNT=0;break;
    default. Encoder_TIM=0;
}
return Encoder_TIM.
}
```

2.5 Bluetooth programming

Bluetooth module is used for the communication between APP and the cart, which can realize the functions of controlling the motion steering of the cart and checking the data, PID parameterization, and saving the parameters by power-down. The Bluetooth module communicates with the cell phone using Bluetooth and with the microcontroller using serial port 3, which is equivalent to a bridge between the cell phone and the microcontroller. Initialization is done by `uart3_init(9600)` of the main function.

The microcontroller receives the data of the Bluetooth APP serial port 3 interrupt service function is implemented in the location of the `usart3.c` file `void USART3_IRQHandler (void)` function, not expanded here. The microcontroller sends data to the Bluetooth APP in the `main.c` in the `APP_Show()` function, not to expand here, in the Bluetooth APP's help description of the use of detailed methods.

The following explains how to control the steering of the car after

```
void Get_RC(void)
{
    static u8 TurnFlag=0;
    static float turn;
    //Receive the app control signal, read the
    value in the machine 1 time
    if((Flag_Right||Flag_Left)&&TurnFlag==0)
    {
        turn = balance_point; TurnFlag
        = 1;
    }
    //Left/Right control
    if (Flag_Left)        balance_point = turn + LeftControl;
    else if (Flag_Right)  balance_point = turn - RightControl;
    //APP no control, mechanical median reset
    if(Flag_Right==0&&Flag_Left==0&&TurnFlag==1)
    {
        balance_point = turn; TurnFlag
        = 0;
    }
}
```

receiving the Bluetooth **APP** data, and its function is implemented as follows:

Controlling the steering of the trolley is realized by modifying the mechanical median value of the trolley, when the app receives the number of steering

According to the data, the program will first save the current mechanical median value of the trolley, and then determine which side needs to be steered, add the deviation of the corresponding direction to the original mechanical median value, and then complete the steering; after there is no app control, restore the median value to the original value, and then the steering can be stopped.

2.6 ADC Acquisition of Battery Voltage with Potentiometer Programming

The analog acquisition of the cart is done by ADC1, in which the initialization of ADC is done by `Adc_Init()` in the main function.

Configuration of the specific process is not expanded here, the result of the configuration for the ADC data right-aligned, non-scanning mode, non-continuous conversion mode, triggered by the software acquisition.

```
u16 Get_Adc(u8 ch)
{
    //set the specified ADC's regular group of channels, a sequence, sample time
    ADC-RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5);
    ADC_SoftwareStartConvCmd(ADC1, ENABLE); //enable the software conversion start
    function of the specified ADC1 while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
    //wait for the conversion to finish. //Wait for the conversion to
    finish
    return ADC_GetConversionValue(ADC1); //returns the result of the most recent ADC1
    rule group conversion
}
```

In order to complete the multi-channel acquisition without using the scan mode at the same time, it is necessary to provide an interface to the user, and then open the corresponding channel scanning when the channel to be acquired is passed in, the function implementation process is as follows:

The realization principle of this function is: in the non-scanning mode of the ADC, pass in the channel that the user needs to collect, and the function will set the channel as the collection sequence 1, and then the software triggers to turn on the conversion and return the result. Therefore, the

acquisition of two potentiometers and battery voltage only need to pass in the corresponding channel.

Battery voltage acquisition results versus actual battery voltage:

As seen in [section 1.7](#), the input voltage is divided by series 10k and 1k resistors and connected to the acquisition pins of the microcontroller. From a simple circuit analysis, we can see that the series circuits have equal currents, and the internal voltage relationship is as follows: $U_{\text{实际电压}} = 11 * U_{\text{ADC}}$. Therefore, the battery voltage can be obtained by multiplying the collected voltage by 11 times. Since the data register of the ADC is 12-bit, the maximum voltage it can measure is 3.3V, so the final battery voltage is calculated as follows:

$$U_{\text{实际电压}} = \text{Get_Adc}(\text{Battery_Ch}) * 3.3 * 11 / 4096$$

The principle of the remaining two potentiometers is the same as that of the battery voltage acquisition, and the adjustment of the mechanical

Servo Adjust: $\text{SERVO_BIAS} = (\text{Get_Adc}(\text{Adjust_Ch}) - 1024) / 12$.

Mechanical Median Adjustment: $\text{BalancePoint_Offset} = (5.06f / 1024.0f) * (\text{Get_Adc}(\text{ZhongZhi_Ch}) - 1024);$

median and the initial position of the servo is accomplished through a linear relationship, which is implemented in the program as follows:

3. Trolley Control Flowchart

The flowchart of the trolley program can be divided into two parts, the first is the `while(1)` dead loop part of the main program, and the other is the interrupt processing data and control part. The main program program flowchart is shown in Figure 3-1. After the system is powered on, it will initialize all the peripherals on the cart, including delay function configuration, initializing the corresponding timer to realize the corresponding function, ADC initialization, analog SPI initialization, analog IIC initialization, MPU6050 initialization and DMP library initialization, etc. After the initialization is completed, it enters the `while(1)` dead loop section, and the other is the interrupt processing data and control section. After initialization, it enters into `while(1)` dead loop, in which it will judge whether to send `app` data and refresh OLED or send data to upper computer to display waveforms by the flag bit. (This flag bit is controlled by the key).

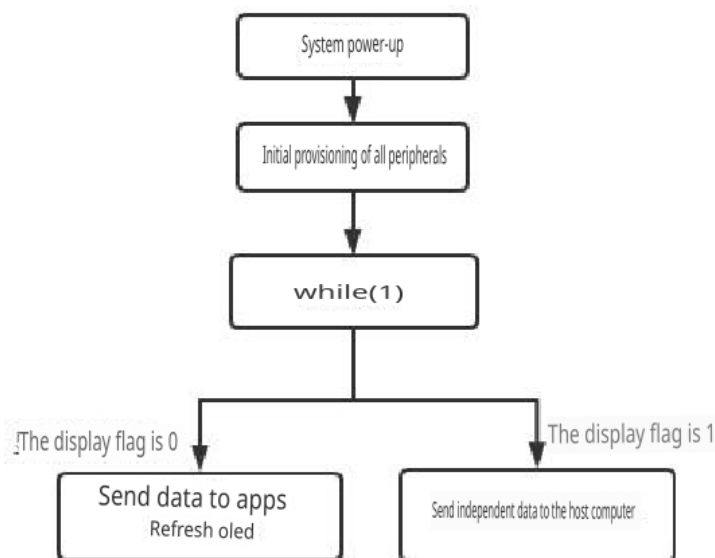


Figure 3-1 Main program program flow chart

The control of the cart is done in interrupts, which are provided by the INT interrupt pin of the MPU6050. The interrupt frequency can be changed by configuring the sampling frequency of the MPU6050. In this program, the interrupt frequency is 200Hz (1 interrupt in 5ms). The control flowchart is shown in Figure 3-2. The control part includes data

reading of all sensors, calculation of **PID+LQR**, control of motor and servo, and receiving data from Bluetooth app.

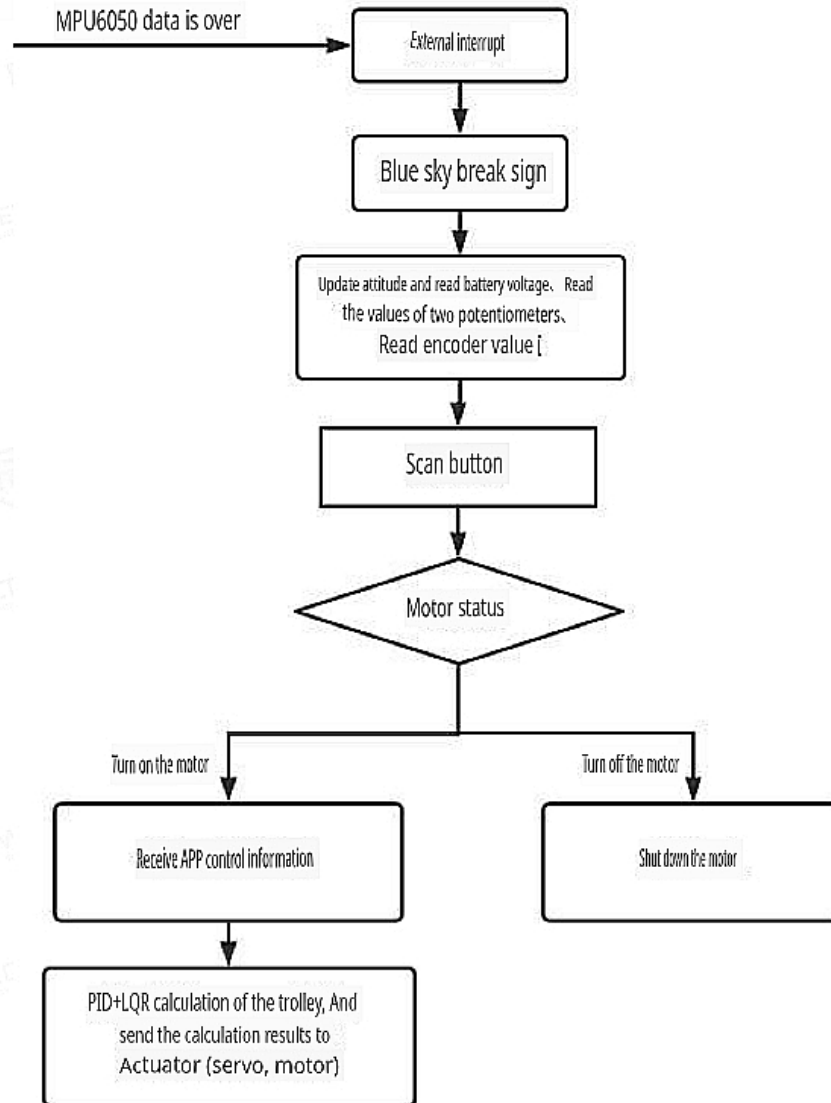


Figure 3-2 Interrupt Task Program Flowchart